

1. Staff → Edit → Update User: the API flow

On page load

```
GET /users/{user_id} → the sectioned profile; fill every form field from it
GET /roles → for the Role dropdown
GET /users → for the Reporting Manager dropdown
```

When the user picks a photo or a document

```
POST /files/upload multipart/form-data, field name: file
→ { file_id, url, name, content_type, size }
```

Hold the returned `url` in frontend state. **Do not save yet** — nothing is attached to the employee at this point. Upload as many files as you need; each call returns its own URL.

When "Update User" is clicked — one single PATCH

```
PATCH /users/{user_id}
{
  "basic information": { "first name": "...", "profile photo": "<url from
upload>" },
  "contact information": { "official email": "...", "mobile number": "..." },
  "address information": { "city": "...", "pin zip code": "..." },
  "employment information": { "employee status": "active", "date_of_joining":
"2026-08-09",
  "shift": "Day Shift", "reporting manager id": "...",
  "role id": "..." },
  "login security": { "username": "...", "password": "...",
"confirm password": "..." },
  "payroll information": { "basic salary": 35000, "bank name": "...", "upi id":
"..." },
  "documents": { "identity proof type": "aadhaar",
  "identity proof file": "<url>",
  "experience certificates": ["<url>", "<url>"] },
  "professional information": { "skills": ["Sales", "CRM"] },
  "system preferences": { "language": "English", "time zone": "Asia/Kolkata",
  "account status": "active" }
}
```

Send only the sections the user actually touched — everything else is left untouched. The response is the same sectioned profile, so you can re-populate the form directly from it.

Add Staff uses the exact same body on `POST /users`. Only two fields are mandatory:

`contact_information.official_email` and `login_security.password`.

Removing things

```
Empty a named slot    PATCH /users/{id}    { "documents": { "resume_cv": null } }
Empty a whole list    PATCH /users/{id}    { "documents": { "experience_certificates":
[] } }
Drop one file          DELETE
/users/{id}/documents/experience_certificates?document_id=<file_id>
Delete the employee    DELETE /users/{id}
Discard an upload      DELETE /files/{file_id}
```

Your 404 is because the frontend is still calling `POST /users/{id}/files/profile_photo`, which no longer exists. Replace it with `POST /files/upload` + `PATCH /users/{id}`.

2. Before vs. Now

Before (old flow — up to 8 calls, uploads tied to the record)

```
1. GET /users/{id} → flat object, all fields
top-level
2. GET /users/meta/employee-options → backend served the dropdown
lists
3. POST /users/{id}/files/profile_photo multipart → saved immediately
4. POST /users/{id}/identity-proof multipart → saved immediately
5. POST /users/{id}/documents/experience_certificates multipart → appended
immediately
6. PATCH /users/{id} → flat body { first name,
phone, ... }
7. PATCH /users/{id}/status { "is_active": false }
PATCH /users/{id}/account-status { "status": "suspended" }
8. PATCH /users/{id}/role { "role_id": "..."} 
```

Problems with that:

- **Uploads needed the employee to already exist.** On the Add Staff screen there was no `user_id` yet, so files could not be uploaded before creating the record.
- Each file save was its own immediate write — a half-filled form could already have written files to the record.
- Status, account status and role each needed a separate call.
- The backend decided which fields were mandatory, and served the dropdown values.

Now (new flow — 1 GET + N uploads + 1 PATCH)

```
1. GET /users/{id} → sectioned profile
2. POST /files/upload multipart → { file_id, url, name, content_type,
size }
(repeat per file; no user_id required, works before the employee exists)
```

```
3. PATCH /users/{id} → one sectioned body: profile + files + status + role + password, all together
```

What changed:

- **One universal upload API**, decoupled from any record. Same endpoint for employees, customers, org logo — anything.
- **One PATCH for everything**. Status change, account status, role reassignment and password change all ride in the same body. A resignation is one call:

```
PATCH /users/{id}
{ "employment_information": { "employee_status": "resigned", "date_of_exit": "2026-08-31" },
  "system_preferences": { "account_status": "inactive" } }
```

- `account_status` drives login — anything other than `active` blocks it.
- **Nothing saves until you press the button**. Uploads produce URLs only; the record is written by the PATCH.
- **The frontend owns validation and dropdowns**. The backend enforces only `official_email` + `password` on create.
- Request and response are the same shape, so read → edit → write round-trips cleanly.

3. Which upload APIs were removed

All four were employee-specific uploads that required a `user_id`:

Removed	What it did	Use instead
POST /users/{id}/files/{field}	Uploaded one slot: <code>profile_photo</code> , <code>identity_proof_file</code> , <code>resume_cv</code> , <code>offer_letter</code> , <code>appointment_letter</code>	POST /files/upload → PATCH /users/{id} with the URL
DELETE /users/{id}/files/{field}	Emptied one slot	PATCH /users/{id} with that field set to <code>null</code>
POST /users/{id}/identity-proof	Legacy ID-proof upload	POST /files/upload → documents.identity_proof_file
POST /users/{id}/documents/{collection}	Multi-file upload for experience / educational / other certificates	POST /files/upload per file → the URLs in documents.<list>

Non-upload routes removed at the same time: GET /users/{id}/documents/{collection}, PATCH /users/{id}/status, PATCH /users/{id}/account-status, PATCH /users/{id}/role, GET /users/meta/employee-options.

What is left on /users – 7 routes

```
POST    /users
GET      /users
GET      /users/{user_id}
PATCH   /users/{user_id}
DELETE   /users/{user_id}
DELETE   /users/{user_id}/documents/{collection}?document_id=<file_id>
POST     /users/{user_id}/reset-password
```

And the file routes – 3

```
POST     /files/upload      (universal; no record id)
GET       /files/{file_id}   (serves the file – put this URL straight in <img src>)
DELETE    /files/{file_id}   (discard an upload that was never attached)
```